

GESTION DE STOCK MULTI-ENTREPÔTS

GESTOCK

Système d'information multi-couches
pour la gestion intelligente des stocks

Architecture en trois couches découplées — Base de données MySQL avec procédures stockées, API REST PHP 8, Application Desktop C# WinForms et Site Web PHP MVC — hébergée en environnement Docker simulé en intranet.

Sommaire

1. Présentation et Objectifs du Projet	3
1.1 Contexte métier	3
1.2 Objectifs pédagogiques	3
1.3 Périmètre fonctionnel	3
2. Architecture Technique Globale	4
2.1 Vue d'ensemble des trois couches	4
2.2 Flux de données et communication	4
2.3 Infrastructure Docker	5
3. Spécifications Fonctionnelles	6
3.1 Application Desktop C# WinForms	6
3.2 Site Web PHP MVC (catalogue public)	7
3.3 API REST PHP 8 (couche intermédiaire)	8
4. Modèle de Données et Logique Métier	9
4.1 Modèle Relationnel — Tables principales	9
4.2 Procédures stockées	11
4.3 Vues SQL	11
5. Livrables et Déploiement	12
5.1 Structure du dépôt GitHub	12
5.2 Configuration Docker Compose	12
5.3 Scripts d'initialisation de la base de données	13

Présentation et Objectifs du Projet

1.1 Contexte Métier

Le projet **Gestock** répond à un besoin concret de nombreuses entreprises de distribution et de logistique : disposer d'un outil fiable permettant de **gérer, en temps réel, les niveaux de stock sur plusieurs entrepôts**, de tracer l'intégralité des mouvements (entrées, sorties, transferts) et d'exposer un catalogue produit à leurs clients internes ou externes.

Dans ce scénario, une entreprise fictive exploite **plusieurs sites de stockage physiques** (entrepôts) et souhaite unifier la gestion de ses références articles, de ses fournisseurs et de la disponibilité par entrepôt au sein d'une plateforme unique, sécurisée et accessible depuis le réseau interne.

PROBLÉMATIQUE CENTRALE

Comment garantir l'**intégrité des données de stock** (pas de valeur négative, cohérence transactionnelle) tout en offrant des interfaces variées (outil employé, catalogue web) sans jamais exposer directement la base de données ?

1.2 Objectifs Pédagogiques

Ce projet de niveau **BTS SIO option SLAM** a pour ambition de démontrer la maîtrise des compétences suivantes, issues du référentiel officiel :

Bases de Données Avancées

- Modélisation relationnelle normalisée (3NF)
- Procédures stockées avec gestion transactionnelle
- Vues SQL pour l'abstraction des données
- Contraintes d'intégrité référentielle

Développement d'API

- Architecture REST avec PHP 8
- Validation des entrées et gestion des erreurs HTTP
- Retours JSON standardisés
- Séparation stricte des responsabilités

Application Desktop

- Développement C# avec WinForms
- Consommation d'une API REST via HttpClient
- Gestion CRUD complète avec UX soignée

Développement Web MVC

- Architecture MVC stricte en PHP
- Séparation Controllers / Models / Views
- Catalogue dynamique en lecture seule

- Découplage total de la couche données

- Consommation de l'API ou des Vues SQL

1.3 Périmètre Fonctionnel

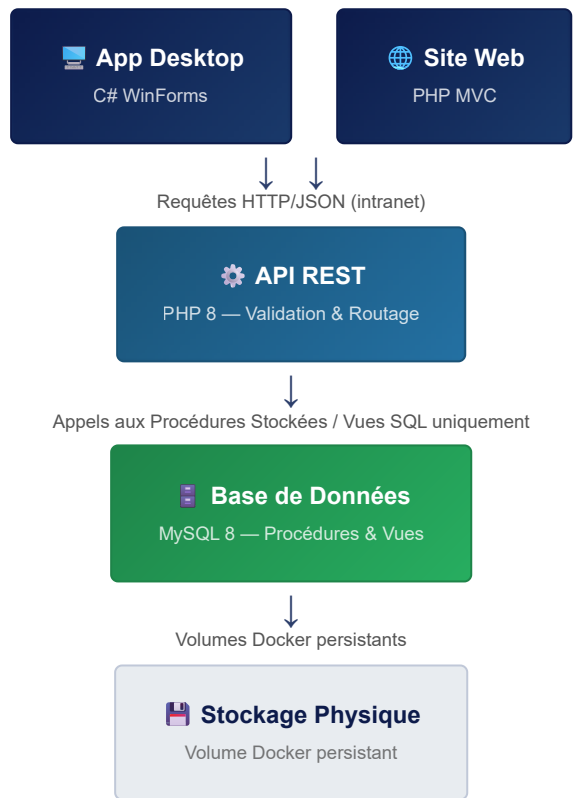
Le système Gestock couvre les domaines fonctionnels suivants :

DOMAINE	FONCTIONNALITÉS COUVERTES	INTERFACE(S) CONCERNÉE(S)	
Référentiel Articles	Création, modification, suppression sécurisée, catégorisation, liaison fournisseur	APP C#	API
Gestion des Entrepôts	CRUD entrepôts, visualisation des stocks par site, alertes de seuil bas	APP C#	API
Mouvements de Stock	Entrées, sorties, historique complet, protection contre le stock négatif	APP C#	API
Catalogue Public	Affichage produits disponibles, filtres par catégorie, disponibilité par entrepôt	SITE WEB	API
Fournisseurs	CRUD fournisseurs, association aux produits	APP C#	API
Alertes Stock	Vue automatique des articles sous le seuil minimal	APP C#	SITE WEB

Architecture Technique Globale

2.1 Vue d'ensemble — Architecture Trois Couches

Gestock repose sur une **architecture en trois couches strictement découplées**, garantissant l'isolation des responsabilités, la maintenabilité et la sécurité de l'ensemble du système. Aucune couche cliente ne communique directement avec la base de données en écriture.



PRINCIPE FONDAMENTAL DE SÉCURITÉ

Aucune application cliente ne peut exécuter de requêtes SQL directes en écriture (INSERT/UPDATE/DELETE). Toutes les opérations de modification passent obligatoirement par les **procédures stockées** de la base de données, appelées via l'API REST. Ce choix centralise la logique métier et élimine les risques d'injection SQL côté client.

2.2 Flux de Données et Communication

ÉMETTEUR	PROTOCOLE	RÉCEPTEUR	FORMAT	TYPE D'OPÉRATION
App C# WinForms	HTTP/REST (intranet)	API PHP 8	JSON	Lecture + Écriture (CRUD complet)

ÉMETTEUR	PROTOCOLE	RÉCEPTEUR	FORMAT	TYPE D'OPÉRATION
Site Web PHP MVC	HTTP/REST (intranet) ou appel direct Vues SQL	API PHP 8 / BDD	JSON / Resultset SQL	Lecture seule (catalogue)
API PHP 8	PDO/MySQL	MySQL 8	Procédures stockées + Vues	Lecture + Écriture (via stored procs)
MySQL 8	Volume Docker	Stockage physique	Fichiers InnoDB	Persistance transactionnelle

Format des réponses API

Toutes les réponses de l'API sont en **JSON** avec une structure normalisée : un champ `status` (`"success"` ou `"error"`), un champ `data` contenant la charge utile, et un champ `message` en cas d'erreur. Les codes HTTP standard sont respectés (200, 201, 400, 404, 409, 500).

2.3 Infrastructure Docker

L'environnement de développement et de démonstration est entièrement containerisé via **Docker Compose**, simulant un réseau intranet. Trois conteneurs principaux sont définis :

CONTENEUR	IMAGE DE BASE	PORT EXPOSÉ	RÔLE
<code>gestock-db</code>	mysql:8.0	3306 (interne)	Base de données MySQL — stockage de toutes les données applicatives
<code>gestock-api</code>	php:8.2-apache	8080	API REST — point d'entrée unique pour toutes les opérations d'écriture
<code>gestock-web</code>	php:8.2-apache	80	Catalogue public — site PHP MVC accessible depuis le réseau interne

RÉSEAU DOCKER INTERNE

Les trois conteneurs partagent un réseau bridge Docker nommé `gestock-network`. Le conteneur `gestock-db` n'expose **aucun port vers l'hôte** : il est uniquement accessible depuis les conteneurs `gestock-api` et `gestock-web` sur le réseau interne, renforçant l'isolation de la couche données.

Spécifications Fonctionnelles

3.1 Application Desktop — C# WinForms

L'application desktop constitue l'**outil de gestion interne** destiné aux employés et responsables logistiques de l'entreprise. Elle est déployée sur les postes de travail du réseau intranet et communique **exclusivement avec l'API REST** via la classe `HttpClient` du framework .NET.

MODULE	FONCTIONNALITÉS	MÉTHODES API APPELÉES
Gestion Produits	Lister, rechercher, créer, modifier, supprimer (suppression sécurisée)	GET /products, POST /products, PUT /products/{id}, DELETE /products/{id}
Gestion Catégories	Créer, modifier, supprimer des catégories d'articles	GET /categories, POST /categories, PUT /categories/{id}, DELETE /categories/{id}
Gestion Fournisseurs	CRUD complet sur les fournisseurs, association aux produits	GET /suppliers, POST /suppliers, PUT /suppliers/{id}, DELETE /suppliers/{id}
Gestion Entrepôts	CRUD entrepôts, visualisation des stocks par entrepôt	GET /warehouses, POST /warehouses, GET /warehouses/{id}/stock
Mouvements de Stock	Enregistrer entrées et sorties, visualiser l'historique complet	POST /stock/in, POST /stock/out, GET /stock/movements
Tableau de Bord	Aperçu des alertes de stock bas, statistiques globales par entrepôt	GET /stock/low, GET /products/available

Exigences techniques de l'application C#

- Développée en **C# .NET 6+** avec l'interface graphique **Windows Forms (WinForms)**.
- Usage de `HttpClient` avec sérialisation/désérialisation JSON via `System.Text.Json` ou `Newtonsoft.Json`.
- Gestion des erreurs HTTP (codes 4xx, 5xx) avec affichage de messages utilisateur explicites.
- Aucune chaîne de connexion à la base de données présente dans le projet C#.
- Validation de formulaires côté client avant envoi à l'API (champs obligatoires, types).
- Architecture interne en couches : **Forms (UI)** → **Services (logique appel API)** → **Models (DTOs)**.

3.2 Site Web — Catalogue Public PHP MVC

Le site web est un **catalogue dynamique en lecture seule**, accessible depuis le réseau intranet. Il présente les produits disponibles, leurs caractéristiques et leur disponibilité par entrepôt. Il **n'expose aucune fonctionnalité de modification** et ne possède pas d'interface d'administration.

PAGE / VUE	DESCRIPTION	SOURCE DE DONNÉES
Accueil	Présentation générale, produits mis en avant, liens vers les catégories	Vue SQL <code>ViewAvailableProducts</code>
Catalogue Produits	Liste paginée avec filtres par catégorie et recherche par nom/référence	API GET /products ou Vue SQL directe
Fiche Produit	Détail d'un article : description, fournisseur, stock par entrepôt	API GET /products/{id} + GET /warehouses/{id}/stock
Stocks Bas	Liste des articles en alerte de rupture (usage interne visible par les employés)	Vue SQL <code>ViewLowStock</code>
Catégories	Navigation par catégorie avec compteur de produits disponibles	API GET /categories

Architecture MVC du site web

Controllers/

Réception des requêtes HTTP, appel du Model approprié, transmission des données à la View. Un contrôleur par domaine fonctionnel : `ProductController`, `CategoryController`, `WarehouseController`.

Models/

Couche d'accès aux données : appels à l'API REST (cURL/file_get_contents) ou requêtes SELECT sur les Vues SQL. Aucune requête INSERT/UPDATE/DELETE dans ce composant.

Views/

Fichiers de templates PHP purs, sans logique métier. Responsables uniquement de l'affichage HTML. Utilisation de `htmlspecialchars()` systématique pour la protection XSS.

Router/

Routeur frontal (index.php + .htaccess) qui dispatche les URLs propres vers le contrôleur approprié. Ex : `/catalogue/produits` → `ProductController::index()`

3.3 API REST — PHP 8 (Couche Intermédiaire)

L'API REST est le **gardien central** du système. Elle est le seul composant autorisé à appeler les procédures stockées MySQL en écriture. Elle valide chaque requête entrante avant de la transmettre à la base de données.

ENDPOINT	MÉTHODE	DESCRIPTION	PROCÉDURE APPELÉE
<code>/products</code>	GET	Retourne la liste des produits (via vue SQL)	<code>ViewAvailableProducts</code>
<code>/products</code>	POST	Crée un nouveau produit après validation	<code>CreateProduct</code>

ENDPOINT	MÉTHODE	DESCRIPTION	PROCÉDURE APPELÉE
<code>/products/{id}</code>	PUT	Met à jour les informations d'un produit existant	<code>UpdateProduct</code>
<code>/products/{id}</code>	DELETE	Supprime un produit si aucun stock actif ne le référence	<code>DeleteProductSafe</code>
<code>/stock/in</code>	POST	Enregistre une entrée de stock dans un entrepôt donné	<code>UpdateStock (+)</code>
<code>/stock/out</code>	POST	Enregistre une sortie de stock (rejet si résultat négatif)	<code>UpdateStock (-)</code>
<code>/stock/movements</code>	GET	Historique paginé des mouvements de stock	SELECT sur <code>stock_movements</code>
<code>/stock/low</code>	GET	Produits dont le stock est sous le seuil minimal	<code>ViewLowStock</code>
<code>/warehouses</code>	GET / POST	Liste ou création d'entrepôts	<code>CreateWarehouse</code>
<code>/warehouses/{id}/stock</code>	GET	Stock consolidé d'un entrepôt spécifique	SELECT sur <code>warehouse_stock</code>
<code>/suppliers</code>	GET / POST / PUT / DELETE	CRUD complet sur les fournisseurs	<code>CreateSupplier</code> , <code>UpdateSupplier</code>

Règles de validation API

- Vérification du **Content-Type** (`application/json` obligatoire pour POST/PUT).
- Contrôle de la présence et du type de chaque champ requis (non vide, entier positif, longueur max).
- Retour du code HTTP **400 Bad Request** avec détail des erreurs si la validation échoue.
- Retour du code HTTP **409 Conflict** si l'opération est métier impossible (stock insuffisant).
- Toute exception PHP non catchée remonte un code HTTP **500** avec message générique (jamais de stack trace en production).

Modèle de Données et Logique Métier

4.1 Modèle Relationnel — Tables Principales

La base de données MySQL 8 utilise le moteur **InnoDB** pour toutes les tables, garantissant le support des transactions ACID et des contraintes de clé étrangère. La base est nommée `gestock_db`.

Table `categories` — Catégories de produits

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique de la catégorie
name	VARCHAR(100)	NOT NULL, UNIQUE	Nom de la catégorie (ex : "Électronique")
description	TEXT	NULL	Description optionnelle de la catégorie
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date de création

Table `suppliers` — Fournisseurs

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique du fournisseur
name	VARCHAR(150)	NOT NULL	Raison sociale du fournisseur
contact_email	VARCHAR(200)	NULL	Adresse email de contact
phone	VARCHAR(20)	NULL	Numéro de téléphone
address	TEXT	NULL	Adresse postale complète
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date de création

Table `products` — Référentiel Produits

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique du produit
reference	VARCHAR(50)	NOT NULL, UNIQUE	Code article interne unique
name	VARCHAR(200)	NOT NULL	Désignation commerciale du produit
description	TEXT	NULL	Description détaillée
unit_price	DECIMAL(10,2)	NOT NULL, ≥ 0	Prix unitaire HT
min_stock_level	INT UNSIGNED	DEFAULT 0	Seuil de stock minimum déclenchant l'alerte

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
category_id	INT UNSIGNED	FK → categories.id	Catégorie du produit
supplier_id	INT UNSIGNED	FK → suppliers.id	Fournisseur principal
is_active	TINYINT(1)	DEFAULT 1	Indicateur de produit actif (soft delete)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date de création
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Date de dernière modification

Table `warehouses` — Entrepôts

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique de l'entrepôt
name	VARCHAR(150)	NOT NULL, UNIQUE	Nom de l'entrepôt (ex : "Entrepôt Nord Lyon")
location	VARCHAR(250)	NULL	Adresse ou localisation géographique
capacity	INT UNSIGNED	NULL	Capacité maximale en unités (optionnel)
is_active	TINYINT(1)	DEFAULT 1	Entrepôt actif ou archivé
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date de création

Table `warehouse_stock` — Stock Multi-Entrepôts (table pivot)

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique de la ligne de stock
product_id	INT UNSIGNED	FK → products.id, NOT NULL	Produit concerné
warehouse_id	INT UNSIGNED	FK → warehouses.id, NOT NULL	Entrepôt concerné
quantity	INT UNSIGNED	NOT NULL, DEFAULT 0	Quantité actuellement en stock (≥ 0 garanti par la procédure)
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Dernière mise à jour du stock

CONTRAINTE D'UNICITÉ

La combinaison `(product_id, warehouse_id)` est soumise à une contrainte UNIQUE. Un produit ne peut donc avoir **qu'une seule ligne de stock par entrepôt**. La procédure `UpdateStock` gère l'insertion initiale (si la ligne n'existe pas) ou la mise à jour.

Table `stock_movements` — Historique des Mouvements

COLONNE	TYPE	CONTRAINTE	DESCRIPTION
id	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant unique du mouvement
product_id	INT UNSIGNED	FK → products.id, NOT NULL	Produit concerné par le mouvement
warehouse_id	INT UNSIGNED	FK → warehouses.id, NOT NULL	Entrepôt concerné
movement_type	ENUM('IN','OUT')	NOT NULL	Type : entrée (IN) ou sortie (OUT) de stock
quantity	INT UNSIGNED	NOT NULL	Quantité déplacée (toujours positive)
reason	VARCHAR(255)	NULL	Motif du mouvement (commande, retour, inventaire...)
operator	VARCHAR(100)	NULL	Identifiant de l'employé ayant effectué l'opération
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date et heure exactes du mouvement

4.2 Procédures Stockées

Les procédures stockées centralisent la **logique métier critique** directement dans la base de données. Cette approche garantit que les règles de gestion s'appliquent *quelle que soit* la source de l'appel, tout en protégeant l'intégrité des données.

PROCÉDURE	PARAMÈTRES	LOGIQUE MÉTIER IMPLÉMENTÉE
<code>CreateProduct</code>	name, reference, price, category_id, supplier_id, min_stock	Vérifie l'unicité de la référence, insère le produit et retourne son nouvel ID
<code>UpdateProduct</code>	product_id, name, price, category_id, supplier_id, min_stock	Met à jour les informations, met à jour <code>updated_at</code>
<code>DeleteProductSafe</code>	product_id	Vérifie que le stock total du produit est à 0 sur tous les entrepôts. Si oui : désactivation (soft delete <code>is_active = 0</code>). Si non : retourne un signal d'erreur SQL (SIGNAL SQLSTATE) sans suppression.
<code>UpdateStock</code>	product_id, warehouse_id, quantity, movement_type, reason, operator	Cœur transactionnel du système : - Démarre une transaction BEGIN. - Si <code>movement_type = 'OUT'</code> : vérifie que <code>quantity ≤ stock actuel</code>. Si non : ROLLBACK + SIGNAL. - Applique la variation (+/-) dans <code>warehouse_stock</code> (INSERT ON DUPLICATE KEY UPDATE). - Insère un enregistrement dans <code>stock_movements</code>. - COMMIT.
<code>CreateWarehouse</code>	name, location, capacity	Crée un entrepôt après vérification d'unicité du nom, retourne l'ID créé
<code>CreateSupplier</code>	name, contact_email, phone, address	Insère un nouveau fournisseur dans le référentiel

PROCÉDURE	PARAMÈTRES	LOGIQUE MÉTIER IMPLÉMENTÉE
<code>UpdateSupplier</code>	supplier_id, name, contact_email, phone, address	Met à jour les coordonnées d'un fournisseur existant

RÈGLE D'OR — INTÉGRITÉ DU STOCK

La procédure `UpdateStock` est la **seule voie autorisée** pour modifier les quantités dans `warehouse_stock`. Son bloc transactionnel garantit qu'un stock ne peut jamais devenir négatif et que chaque mouvement est systématiquement tracé dans l'historique. Un trigger BEFORE UPDATE pourra être ajouté comme filet de sécurité supplémentaire.

4.3 Vues SQL

Les vues SQL fournissent des **abstractions de lecture pré-calculées** que les applications peuvent consommer sans connaître la structure interne des tables.

VUE SQL	TABLES UTILISÉES	DONNÉES RETOURNÉES	USAGE
<code>ViewAvailableProducts</code>	products, categories, suppliers, warehouse_stock	Tous les produits actifs avec leur catégorie, fournisseur et stock total cumulé sur tous les entrepôts	Catalogue public, liste produits App C#
<code>ViewLowStock</code>	products, warehouse_stock, warehouses	Produits dont le stock total est inférieur ou égal à <code>min_stock_level</code> , avec détail par entrepôt	Alertes tableau de bord App C#, page alertes Site Web
<code>ViewStockByWarehouse</code>	warehouse_stock, products, warehouses	Liste des quantités par produit et par entrepôt, avec le nom de l'entrepôt et du produit	Visualisation de l'état des entrepôts dans l'App C#
<code>ViewRecentMovements</code>	stock_movements, products, warehouses	50 derniers mouvements avec toutes les informations de contexte dénormalisées	Historique dans l'App C# et tableau de bord

Livrables et Déploiement

5.1 Structure du Dépôt GitHub

Le projet est versionné sur un unique dépôt GitHub public, organisé en répertoires correspondant à chaque composant du système. Cette structure facilite la compréhension de l'architecture multi-couches par les évaluateurs.

STRUCTURE DU DÉPÔT

```
gestock/
├── docker-compose.yml — Orchestration des 3 conteneurs
├── README.md — Guide d'installation et de démarrage
├──
├── database/
│   ├── schema.sql — Création des tables et contraintes
│   ├── procedures.sql — Définition des procédures stockées
│   ├── views.sql — Définition des vues SQL
│   └── data_sample.sql — Données de démonstration
├──
├── api/ — API REST PHP 8
│   ├── Dockerfile
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── middleware/ — Validation des entrées
│   │   └── config/ — Configuration PDO
│   └── index.php — Router frontal
├──
├── webapp/ — Site Web PHP MVC
│   ├── Dockerfile
│   └── src/
│       ├── controllers/
│       ├── models/
│       ├── views/
│       └── router.php
├──
└── desktop-app/ — Application C# WinForms
    ├── Gestock.sln
    ├── Gestock/
    └── Forms/
```

Services/
Models/

5.2 Configuration Docker Compose

Le fichier `docker-compose.yml` à la racine du projet orchestre les trois conteneurs. Il définit les variables d'environnement, les ports, les volumes et le réseau interne.

SERVICE DOCKER	PARAMÈTRE CLÉ	VALEUR	RÔLE
gestock-db	image	mysql:8.0	Moteur de base de données
	MYSQL_DATABASE	gestock_db	Nom de la base créée au démarrage
	volumes	gestock_db_data:/var/lib/mysql	Persistance des données entre redémarrages
	networks	gestock-network	Réseau interne uniquement (pas de port exposé)
gestock-api	build	./api	Image construite depuis le Dockerfile local
	ports	8080:80	API accessible sur le port 8080 de l'hôte
	depends_on	gestock-db	Démarrage conditionnel après la BDD
gestock-web	build	./webapp	Image construite depuis le Dockerfile local
	ports	80:80	Site accessible sur le port 80 de l'hôte
	depends_on	gestock-api	Démarrage après l'API

5.3 Scripts d'Initialisation de la Base de Données

Le dossier `database/` contient l'ensemble des scripts SQL à exécuter dans l'ordre indiqué pour initialiser complètement la base de données. Ces scripts sont également montés dans le conteneur MySQL via le mécanisme `/docker-entrypoint-initdb.d/` pour une initialisation automatique au premier démarrage.

ORDRE	FICHIER	CONTENU	PRÉREQUIS
1	<code>schema.sql</code>	Création de la base <code>gestock_db</code> , création des 6 tables avec toutes les contraintes (PK, FK, UNIQUE, DEFAULT, CHECK), création des index de performance	Aucun
2	<code>procedures.sql</code>	Définition des 7 procédures stockées : <code>CreateProduct</code> , <code>UpdateProduct</code> , <code>DeleteProductSafe</code> , <code>UpdateStock</code> , <code>CreateWarehouse</code> , <code>CreateSupplier</code> , <code>UpdateSupplier</code>	<code>schema.sql</code> exécuté
3	<code>views.sql</code>	Définition des 4 vues SQL : <code>ViewAvailableProducts</code> , <code>ViewLowStock</code> , <code>ViewStockByWarehouse</code> , <code>ViewRecentMovements</code>	<code>schema.sql</code> exécuté

ORDRE	FICHIER	CONTENU	PRÉREQUIS
4	<code>data_sample.sql</code>	Insertion de données de démonstration : 3 catégories, 4 fournisseurs, 2 entrepôts, 10 produits, stocks initiaux et mouvements d'exemple	Tous les scripts précédents

Commandes de démarrage rapide

DÉMARRAGE DE L'ENVIRONNEMENT COMPLET

Pour démarrer l'intégralité de l'environnement Gestock en local, il suffit d'exécuter depuis la racine du projet cloné :

```
docker compose up --build -d
```

Une fois les conteneurs démarrés : **Site Web** → `http://localhost` | **API** → `http://localhost:8080`

L'application C# doit être compilée séparément et configurée avec l'URL `http://localhost:8080` comme base URL de l'API.

Livrables finaux attendus

- ✓ Dépôt GitHub public avec README complet et instructions de démarrage
- ✓ `docker-compose.yml` fonctionnel permettant le démarrage en une commande
- ✓ Scripts SQL complets et commentés (schema, procédures, vues, données de test)
- ✓ API REST PHP 8 documentée (endpoints, paramètres, réponses attendues)
- ✓ Site Web PHP MVC fonctionnel avec architecture MVC claire
- ✓ Application C# WinForms compilée et installable (.exe ou solution Visual Studio)
- ✓ Ce document (CDC) au format PDF, joint au dossier de projet